

Policy based data structure

```
#include <iostream>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_stree_tag,
tree_order_statistics_node_update> ordered_set;
int main()
{
    ordered_set st;
    st.insert(1);
    st.insert(4);
    st.insert(8);
    st.insert(12);
    // 1. Find the 2nd smallest element (0-indexed)
    // Output: 8
    cout << "Value at index 2: " <<
    *st.find_by_order(2) << endl;

    // 2. How many elements are smaller than 8?
    // Output: 2 (elements 1 and 4)
    cout << "Count smaller than 8: " <<
    st.order_of_key(8) << endl;

    return 0;
}
```

Includes

```
#include <iostream>
#include <iomanip>
#include <fstream>
#include <cstdio>
#include <cstdlib>
#include <cassert>
#include <climits>
#include <cmath>
#include <algorithm>
#include <cstring>
#include <string>
#include <vector>
#include <list>
#include <stack>
#include <set>
#include <bitset>
#include <queue>
#include <map>
```

```
#include <sstream>
#include <functional>
#include <unordered_map>
#include <unordered_set>
#include <complex>
#include <random>
#include <chrono>
```

GENERAL CODE SYNTAX**1.Binary search(upper and lower bound)**

```
vector<int> v = {10, 20, 30, 30, 40};
int x = 30;

// First element >= x
auto lb = lower_bound(v.begin(), v.end(), x);
int idx_lb = lb - v.begin(); // Index: 2
```

```
// First element > x
auto ub = upper_bound(v.begin(), v.end(), x);
int idx_ub = ub - v.begin(); // Index: 4
```

```
// Count occurrences of x in O(log N)
int count = ub - lb; // Result: 2
```

```
// Check if x exists
bool exists = binary_search(v.begin(), v.end(), x);
```

2.Built-in Bit Manipulation (GCC)

```
unsigned int n = 7; // Binary: 111
```

```
__builtin_popcount(n); // Count set bits (1s).
Result: 3
```

```
__builtin_clz(n); // Count leading zeros
(starts from most significant bit)
```

```
__builtin_ctz(n); // Count trailing zeros
(starts from least significant bit)
```

```
__builtin_parity(n); // 1 if number of set bits is
odd, 0 if even
```

```
// For long long, append 'll'
__builtin_popcountll(n_ll);
```

3. Floating point precision

```
const double EPS = 1e-9;
if (abs(a - b) < EPS) {
    // a and b are effectively equal
}

// Print with fixed precision
cout << fixed << setprecision(10) << ans << endl;
```

4. String and character utilities

```
// Check character types
isalpha(c); isdigit(c); isalnum(c); islower(c);
isupper(c);
```

```
// Convert case
char lower = tolower(c);
```

```
// String to Number / Number to String
int x = stoi("123");
string s = to_string(123);
```

6. Custom sort with lambdas

```
sort(v.begin(), v.end(), [](const Type &a, const
Type &b) {
    if (a.value != b.value) return a.value < b.value;
    return a.secondary < b.secondary; // Tie-breaker
});
```

7. Character to number and vice versa

```
char c = '7';
int num = c - '0'; // Convert '7' to 7
char back = num + '0'; // Convert 7 back to '7'
char letter = 'd';
int index = letter - 'a'; // 'a' -> 0, 'b' -> 1, 'c' -> 2, 'd' -> 3
```

```
// Convert index back to character
char original = index + 'a';
```

```
// For Uppercase
char cap = 'G';
int cap_idx = cap - 'A';
```

8. Setting and unsettings bits etc

Set i-th bit $n \mid (1 \ll i)$ Uses OR to force the bit to 1.

Unset (Clear) i-th bit $n \& \sim(1 \ll i)$ Uses AND with a mask of all 1s except at position i.

Toggle i-th bit $n \wedge (1 \ll i)$ Uses XOR to flip the bit (0 to 1, or 1 to 0).

$(n \gg i) \& 1$ for checking if ith bit set or not

$n \& (n + 1)$ clears all trailing ones

$n \mid (n + 1)$ sets last cleared bit

$n \& -n$ extracts last set bit

9 Bitwise arithmetic identities

$a + b = (a \oplus b) + 2(a \& b)$

$a + b = (a \mid b) + (a \& b)$

$a \oplus b = (a \mid b) - (a \& b)$

$a \oplus b = (a \mid b) \& \sim(a \& b)$ (This is the literal definition of XOR)

$a + b \geq a \oplus b$

$a \mid b = a \oplus b + a \& b$

$a \oplus (a \& b) = (a \mid b) \oplus b$

$b \oplus (a \& b) = (a \mid b) \oplus a$

$(a \& b) \oplus (a \mid b) = a \oplus b$

10. Customer comparator

```
struct Comp {
    bool operator()(const pair<int,int>& a, const
pair<int,int>& b) const {
        return a.first + a.second < b.first + b.second;
    }
};
```

```
};
```

```
set<pair<int,int>, Comp>;
```

```
struct Node { int u, v, cost; // Overloading the <
operator bool operator<(const Node& other) const
{ if (cost != other.cost) return cost < other.cost;
return u < other.u; // Tie-breaker } };
```

GRAPHS

TopoSort

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;
```

```
void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

Bipartite

```
int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
```

```
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}
```

```
cout << (is_bipartite ? "YES" : "NO") << endl;
```

Dijkstra algorithm

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;
```

```
void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);
    vector<bool> u(n, false);

    d[s] = 0;
    for (int i = 0; i < n; i++) {
        int v = -1;
        for (int j = 0; j < n; j++) {
            if (!u[j] && (v == -1 || d[j] < d[v]))
                v = j;
        }
    }
```

```
    if (d[v] == INF)
        break;
```

```
    u[v] = true;
    for (auto edge : adj[v]) {
        int to = edge.first;
        int len = edge.second;

        if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            p[to] = v;
        }
    }
```

```

    }
}
}

```

Bellman ford

```

struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (Edge e : edges)
            if (d[e.a] < INF)
                d[e.b] = min(d[e.b], d[e.a] + e.cost);
    // display d, for example, on the screen
}

```

Floyd warshall

```

for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

if neg weights then
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

```

Cycle detection directed graph

```

int n;
vector<vector<int>>> adj;

```

```

vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

```

```

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

```

```

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v =
parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

```

    }
}

Cycle detection undirected graph
int n;
vector<vector<int>>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its
parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to
parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v =
parent[v])
            cycle.push_back(v);
    }
}

```

```

        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

MST (Prims)
#include <iostream>
#include <vector>
#include <queue>
#include <limits>

using namespace std;

// Pair representing (weight, destination_node)
typedef pair<int, int> pii;

void primsMST(vector<vector<pii>>& adj, int
startNode, int V) {
    // Min-heap to pick the minimum weight edge
    priority_queue<pii, vector<pii>, greater<pii>>
pq;

    vector<int> key(V, INT_MAX); // Minimum
weight to connect node
    vector<int> parent(V, -1); // To store the
structure of MST
    vector<bool> inMST(V, false); // To track nodes
included in MST

    // Start with the initial node
    pq.push({0, startNode});
    key[startNode] = 0;

    int totalCost = 0;

    while (!pq.empty()) {
        int u = pq.top().second;
        int weight = pq.top().first;
        pq.pop();

        if (inMST[u]) continue; // Skip if node is
already in MST
    }
}

```

```

inMST[u] = true;
totalCost += weight;

// Explore neighbors
for (auto& neighbor : adj[u]) {
    int v = neighbor.first;
    int w = neighbor.second;

    // If v is not in MST and weight of (u,v) is
    smaller than current key
    if (!inMST[v] && w < key[v]) {
        key[v] = w;
        pq.push({key[v], v});
        parent[v] = u;
    }
}

// Print the result
cout << "Edge \tWeight" << endl;
for (int i = 1; i < V; ++i) {
    if (parent[i] != -1)
        cout << parent[i] << " - " << i << " \t" <<
key[i] << endl;
}
cout << "Total Minimum Cost: " << totalCost <<
endl;
}

int main() {
    int V // Number of vertices
    vector<vector<pii>> adj(V);

    // Adding edges: adj[u].push_back({v, weight})
    adj[0].push_back({1, 2}); adj[1].push_back({0,
2});
    adj[0].push_back({2, 3}); adj[2].push_back({0,
3});
    primsMST(adj, 0, V);
    return 0;
}

Investigation (Dijkstra with states)
#include <iostream>
#include <vector>
#include <queue>

#include <algorithm>
using namespace std;
#define ll long long

static const ll MOD = 1e9 + 7;
const ll INF = 1e17;

ll dist[100005];
ll ways[100005];
ll min_f[100005];
ll max_f[100005];
vector<vector<pair<ll, ll>>> adj;

void dijkstra(ll n) {
    // 1. Initialize all arrays
    for (int i = 1; i <= n; i++) {
        dist[i] = INF;
        ways[i] = 0;
        min_f[i] = INF;
        max_f[i] = 0;
    }

    // 2. Base Case: Start at City 1
    dist[1] = 0;
    ways[1] = 1;
    min_f[1] = 0;
    max_f[1] = 0;

    // pair<price, node>
    priority_queue<pair<ll, ll>, vector<pair<ll, ll>>,
greater<pair<ll, ll>>> q;
    q.push({0, 1});

    while (!q.empty()) {
        ll d_u = q.top().first;
        ll u = q.top().second;
        q.pop();

        // 3. Standard Dijkstra check
        if (d_u > dist[u]) continue;

        for (auto& edge : adj[u]) {
            ll v = edge.first;
            ll weight = edge.second;

            // Case A: Found a strictly better price

```

```

        if (dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            ways[v] = ways[u];
            min_f[v] = min_f[u] + 1;
            max_f[v] = max_f[u] + 1;
            q.push({dist[v], v});
        }
        // Case B: Found an alternative path with
the same minimum price
        else if (dist[u] + weight == dist[v]) {
            ways[v] = (ways[v] + ways[u]) % MOD;
            min_f[v] = min(min_f[v], min_f[u] + 1);
            max_f[v] = max(max_f[v], max_f[u] +
1);
        }
    }
}

cout << dist[n] << " " << ways[n] << " " <<
min_f[n] << " " << max_f[n] << endl;
}

void solve() {
    ll n, m;
    if (!(cin >> n >> m)) return;

    adj.assign(n + 1, vector<pair<ll, ll>>());
    for (ll i = 0; i < m; i++) {
        ll a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    dijkstra(n);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // precompute(); // Not needed for this specific
logic, but can keep if used elsewhere
    solve();
    return 0;
}

```

DSU (With CSES Question)

```

#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
public:
    vector<int> parent, sz;
    int numComponents;
    int maxComponentSize;

    DisjointSet(int n) {
        parent.resize(n + 1);
        sz.resize(n + 1, 1); // Initially, each city is a
component of size 1
        numComponents = n; // Initially, there are n
components
        maxComponentSize = 1; // Largest component
starts at 1
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);

        // If they are already in the same component,
do nothing
        if (ulp_u == ulp_v) return;

        // Union by size: attach smaller to larger
        if (sz[ulp_u] < sz[ulp_v]) {
            parent[ulp_u] = ulp_v;
            sz[ulp_v] += sz[ulp_u];
            maxComponentSize =
max(maxComponentSize, sz[ulp_v]);
        } else {
            parent[ulp_v] = ulp_u;
            sz[ulp_u] += sz[ulp_v];

```

```

        maxComponentSize =
max(maxComponentSize, sz[ulp_u]);
    }

    // One successful merge means one less
component
    numComponents--;
    }
};

int main() {
    // Speed up I/O for CP
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    DisjointSet ds(n);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        ds.unionBySize(u, v);

        // Print the two required values after each road
        cout << ds.numComponents << " " <<
ds.maxComponentSize << "\n";
    }

    return 0;
}

SCC
#include <bits/stdc++.h>
using namespace std;

vector<bool> visited;

// DFS to find the finishing order
void dfs(int v, vector<vector<int>> const& adj,
vector<int> &output) {
    visited[v] = true;
    for (auto u : adj[v])
        if (!visited[u])
            dfs(u, adj, output);

    output.push_back(v);
}

// Your provided SCC logic
void strongly_connected_components(int n,
vector<vector<int>> const& adj,
vector<int>
&kingdom_assignment,
int &kingdom_count) {
    vector<int> order;
    visited.assign(n + 1, false);

    // Step 1: First DFS to get finishing times
    for (int i = 1; i <= n; i++)
        if (!visited[i])
            dfs(i, adj, order);

    // Step 2: Reverse the graph
    vector<vector<int>> adj_rev(n + 1);
    for (int v = 1; v <= n; v++) {
        for (int u : adj[v]) {
            adj_rev[u].push_back(v);
        }
    }

    // Step 3: Second DFS on reversed graph in order
of finishing times
    visited.assign(n + 1, false);
    reverse(order.begin(), order.end());

    kingdom_count = 0;
    for (auto v : order) {
        if (!visited[v]) {
            kingdom_count++;
            vector<int> component;
            dfs(v, adj_rev, component);
            // Assign the current kingdom ID to all
planets in this component
            for (auto u : component) {
                kingdom_assignment[u] =
kingdom_count;
            }
        }
    }
}

```

```

int main() {
    // Fast I/O
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
    }

    vector<int> kingdom_assignment(n + 1);
    int total_kingdoms = 0;

    strongly_connected_components(n, adj,
    kingdom_assignment, total_kingdoms);

    // Output the results
    cout << total_kingdoms << "\n";
    for (int i = 1; i <= n; i++) {
        cout << kingdom_assignment[i] << (i == n ?
"" : " ");
    }
    cout << endl;

    return 0;
}

```

MATHS

Spf sieve

```

const int MAXN = 1e6 + 5;
int spf[MAXN];

void sieve() {
    iota(spf, spf + MAXN, 0);
    for (int i = 2; i * i < MAXN; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j < MAXN; j += i)
                if (spf[j] == j) spf[j] = i;
        }
    }
}

```

```

}

// Get prime factorization in O(log n)
vector<int> getFactorization(int x) {
    vector<int> ret;
    while (x != 1) {
        ret.push_back(spf[x]);
        x /= spf[x];
    }
    return ret;
}

```

Normal Sieve

```

int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i * i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}

```

Find primes range

```

vector<char> segmentedSieve(long long L, long
long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }
}

```

```

vector<char> isPrime(R - L + 1, true);
for (long long i : primes)
    for (long long j = max(i * i, (L + i - 1) / i * i); j
<= R; j += i)
        isPrime[j - L] = false;
if (L == 1)
    isPrime[0] = false;
return isPrime;

```

```
}
```

Binary expo with mod

```
long long binpow(long long a, long long b, long
long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

For calc nCr

```
static const long long MOD = 1e9 + 7;
static const int MAXN = 1000000; // adjust as
needed
```

```
long long fact[MAXN + 1], invFact[MAXN + 1];

long long power(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}
```

```
void precompute() {
```

```
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++)
        fact[i] = fact[i - 1] * i % MOD;
```

```
    invFact[MAXN] = power(fact[MAXN], MOD -
2); // Fermat
    for (int i = MAXN - 1; i >= 0; i--)
        invFact[i] = invFact[i + 1] * (i + 1) % MOD;
}
```

```
long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n -
r] % MOD;
}
# if n value very large then
long long lucas(long long n, long long r, int p) {
    if (r == 0) return 1;

    return (lucas(n / p, r / p, p) * nCr(n % p, r % p))
% p;
}
```

```
long long INF = 1e17;
int main() {precompute();solve();}
```

Miller-Rabin (Primality Test for 10^{18})

```
bool miller_rabin(long long n, int a) {
    if (a % n == 0) return true;
    long long d = n - 1;
    while (d % 2 == 0) d /= 2;
    long long x = binpow(a, d, n);
    if (x == 1 || x == n - 1) return true;
    while (d != n - 1) {
        x = (__int128_t)x * x % n;
        d *= 2;
        if (x == n - 1) return true;
        if (x == 1) return false;
    }
    return false;
}
```

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
37}) {
        if (n == a) return true;
        if (!miller_rabin(n, a)) return false;
    }
    return true;
}
```

Pollard's Rho (Factorization for 10^{18})

```

long long pollard_rho(long long n) {
    if (n % 2 == 0) return 2;
    if (isPrime(n)) return n;
    long long x = rand() % (n - 2) + 2;
    long long y = x;
    long long c = rand() % (n - 1) + 1;
    long long d = 1;
    while (d == 1) {
        x = ((__int128_t)x * x + c) % n;
        y = ((__int128_t)y * y + c) % n;
        y = ((__int128_t)y * y + c) % n;
        d = __gcd(abs(x - y), n);
        if (d == n) return pollard_rho(n);
    }
    return d;
}

```

Matrix expo

```

struct Matrix {
    int sz;
    vector<vector<long long>> mat;

    Matrix(int n) : sz(n) {
        mat.assign(n, vector<long long>(n, 0));
    }

    // Identity Matrix
    static Matrix identity(int n) {
        Matrix res(n);
        for (int i = 0; i < n; i++) res.mat[i][i] = 1;
        return res;
    }

    // Matrix Multiplication (O(N^3))
    Matrix operator*(const Matrix& other) const {
        Matrix res(sz);
        for (int i = 0; i < sz; i++) {
            for (int k = 0; k < sz; k++) { // k-loop inside (a1 + a2 + .... ak)
                for (int j = 0; j < sz; j++) {
                    res.mat[i][j] = (res.mat[i][j] + mat[i][k]
* other.mat[k][j]) % MOD;
                }
            }
        }
    }
}

```

```

        return res;
    }
};

// Binary Exponentiation for Matrices (O(N^3 log p))
Matrix power(Matrix a, long long p) {
    Matrix res = Matrix::identity(a.sz);
    while (p > 0) {
        if (p & 1) res = res * a;
        a = a * a;
        p >>= 1;
    }
    return res;
}

long long solve_fib(long long n) {
    if (n == 0) return 0;
    if (n == 1) return 1;

    // 1. Create the transition matrix T = [[1, 1], [1, 0]]
    Matrix T(2);
    T.mat[0][0] = 1; T.mat[0][1] = 1;
    T.mat[1][0] = 1; T.mat[1][1] = 0;

    // 2. Raise T to the power (n - 1)
    T = power(T, n - 1);

    // 3. The result is T^n-1 * [F1, F0]^T
    // Since F1 = 1 and F0 = 0, the result is simply
    T.mat[0][0]
    return T.mat[0][0];
}

```

Stars and Bars

formula: $n+k-1 \text{ C } k-1$ where n are stars (for $x \geq 0$)
 for $x \geq 1$ $n-1$ Combination $k-1$
 for $x \geq a$ $n' + k - 1$ combination $k-1$ where $n' = n - a$

Dynamic Programming

Removal game

```

void solve() {
    int n;
    cin >> n;
    vector<ll> arr(n);
}

```

```

ll sum = 0;
for (ll i = 0; i < n; i++) {
    cin >> arr[i];
    sum += arr[i];
}
vector<vector<ll>>dp(n, vector<ll>(n, 0));
for (ll i = 0; i < n; i++) {
    dp[i][i] = arr[i];
}
for (ll i = n - 1; i >= 0; i--) {
    for (ll j = i + 1; j < n; j++) {
        dp[i][j] = max(arr[i] - dp[i + 1][j], arr[j] -
dp[i][j - 1]);
    }
}
cout << (dp[0][n - 1] + sum) / 2 << endl;
// s1 -s2 = maxi
// s1 + s2 = total

```

Projects

```

struct Project {
    ll start, end, reward;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Project> projects(n);
    for (int i = 0; i < n; ++i) {
        cin >> projects[i].start >> projects[i].end >>
projects[i].reward;
    }

    // Sort projects by end day
    sort(projects.begin(), projects.end(), [](const
Project &a, const Project &b){
        return a.end < b.end;
    });

    // dp[i] = max profit using first i projects (0-
based)
    vector<ll> dp(n, 0);

```

```

    for (int i = 0; i < n; ++i) {
        ll take = projects[i].reward;

        // Binary search for last project that ends
        before projects[i].start
        int lo = 0, hi = i - 1, last = -1;
        while (lo <= hi) {
            int mid = lo + (hi - lo)/2;
            if (projects[mid].end < projects[i].start) {
                last = mid;
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
        if (last != -1) take += dp[last];

        ll notTake = (i == 0) ? 0 : dp[i-1];
        dp[i] = max(take, notTake);
    }

    cout << dp[n-1] << "\n";
    return 0;
}

```

Elevator Rides(Bitmask dp)

```

void solve() {
    int n;
    ll w_limit; // max weight
    if (!(cin >> n >> w_limit)) return;
    vector<ll> weights(n);
    for (int i = 0; i < n; i++) {
        cin >> weights[i];
    }
    // dp[mask] = {rides, weight_of_last_ride}
    // Initialize with n+1 rides (more than possible)
    vector<pair<ll, ll>> dp(1 << n, {n + 1, 0});
    dp[0] = {1, 0}; // Start with 1 ride, 0 weight used

    for (int mask = 0; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {

```

```

// Check if person i is NOT in the mask
if (!(mask & (1 << i))) {
    int nextMask = mask | (1 << i);
    pair<ll, ll> cur = dp[mask];
    // Can person i fit in the current last ride?
    if (cur.second + weights[i] <= w_limit) {
        cur.second += weights[i];
    } else {
        // Start a new ride
        cur.first++;
        cur.second = weights[i];
    }
    // Update nextMask if this result is better
    if (cur < dp[nextMask]) {
        dp[nextMask] = cur;
    }
}

cout << dp[(1 << n) - 1].first << endl;
}

```

Digit dp

```

long long memo[20][2][2][sum_limit]; // pos, tight,
leading_zeros, state

```

```

long long solve(string &s, int pos, bool tight, bool
leading, int state) {
    if (pos == s.size()) return 1; // Base case
    if (memo[pos][tight][leading][state] != -1) return
memo[pos][tight][leading][state];
    int limit = tight ? (s[pos] - '0') : 9;
    long long ans = 0;
    for (int d = 0; d <= limit; d++) {
        ans += solve(s, pos + 1, tight && (d == limit),
leading && (d == 0), new_state);
    }
    return memo[pos][tight][leading][state] = ans;
}

```

MCM Pattern

```

for (int len = 1; len <= n; len++) {
    for (int i = 0; i + len - 1 < n; i++) {
        int j = i + len - 1;
        if (len == 1) { dp[i][j] = base_case; continue; }
        for (int k = i; k < j; k++) {
            dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] +
cost(i, k, j));
        }
    }
}

```

Trees

Building trees from traversals

```

#include <iostream>
#include <vector>
#include <queue>
#include <unordered_map>
using namespace std;
struct Node {
    int data;
    Node *left, *right;

    Node(int val) : data(val), left(nullptr),
right(nullptr) {}
};

```

```

};
class TreeBuilder {
private:
    unordered_map<int, int> inMap;

```

```

Node* buildPreInSubtree(vector<int>& pre, int&
preIdx, int inStart, int inEnd) {
    if (inStart > inEnd) return nullptr;
    int curr = pre[preIdx++];
    Node* root = new Node(curr);
    int pos = inMap[curr];
    root->left = buildPreInSubtree(pre, preIdx, inStart,
pos - 1);
    root->right = buildPreInSubtree(pre, preIdx, pos +
1, inEnd);
    return root;
}

Node* buildPostInSubtree(vector<int>& post,
int& postIdx, int inStart, int inEnd) {
    if (inStart > inEnd) return nullptr
    int curr = post[postIdx--];
    Node* root = new Node(curr);
    int pos = inMap[curr];
    root->right = buildPostInSubtree(post, postIdx, pos +
1, inEnd);
    root->left = buildPostInSubtree(post, postIdx,
inStart, pos - 1);
    return root;
}

public:
Node* fromPreIn(vector<int> pre, vector<int> in)
{
    inMap.clear();
    for(int i = 0; i < in.size(); i++) inMap[in[i]] = i;
    int preIdx = 0;
    return buildPreInSubtree(pre, preIdx, 0, in.size() -
1);
}

Node* fromPostIn(vector<int> post, vector<int>
in) {
    inMap.clear();
    for(int i = 0; i < in.size(); i++) inMap[in[i]] = i;
    int postIdx = post.size() - 1;
    return buildPostInSubtree(post, postIdx, 0,
in.size() - 1);
}

Node* fromLevelOrder(vector<int> level) {
    if (level.empty() || level[0] == -1) return
nullptr;

    Node* root = new Node(level[0]);

    queue<Node*> q;

    q.push(root);

    int i = 1;

    while (i < level.size()) {

        Node* curr = q.front(); q.pop();

        if (i < level.size() && level[i] != -1) {

            curr->left = new Node(level[i]);

            q.push(curr->left);

        }

        i++;

        if (i < level.size() && level[i] != -1)
curr->right = new Node(level[i]);
q.push(curr->right); }
i++; }
return root;
}

void printLevelOrder(Node* root) {
    if (!root) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty())
        Node* curr = q.front(); q.pop();
        cout << curr->data << " ";
    if (curr->left) q.push(curr->left);
    if (curr->right) q.push(curr->right);}cout << endl;
}

void deleteTree(Node* root) {
    if (!root) return;

```

```

deleteTree(root → left);
deleteTree(root → right);
delete root;
}
};

int main() {

    TreeBuilder tb;
    vector<int> inorder = {4, 2, 5, 1, 6, 3, 7}, preorder = {1, 2, 4, 5, 3, 6, 7}, postorder = {4, 5, 2, 6, 7, 3, 1}, levelData = {1, 2, 3, 4, 5, 6, 7};
    cout << "Pre+In: "; Node* r1 = tb.fromPreIn(preorder, inorder);
    tb.printLevelOrder(r1);
    cout << "Post+In: "; Node* r2 = tb.fromPostIn(postorder, inorder);
    tb.printLevelOrder(r2);

    cout << "Level: "; Node* r3 = tb.fromLevelOrder(levelData);
    tb.printLevelOrder(r3);
    tb.deleteTree(r1); tb.deleteTree(r2);
    tb.deleteTree(r3);

    return 0;
}

Kth ancestor(Binary lifting)

#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
using namespace std;
// Using 21 for LOG covers up to 2^20 (approx 1 million nodes)
const int MAXN = 200005;
const int LOG = 21;
int up[MAXN][LOG];
void solve() {
    int n, q, root;
    if (!(cin >> n >> q >> root)) return;
    // Initialize the table with -1 (no ancestor)
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j < LOG; j++) {
            up[i][j] = -1;
        }
        // Since the input defines parent-child relationships,
        // we can fill up[child][0] directly without a DFS.
        for (int i = 0; i < n - 1; i++) {
            int parent, child;
            cin >> parent >> child;
            up[child][0] = parent;
        }
        // Precompute the binary lifting table (Binary Jumps)
        // up[i][j] = the 2^j-th ancestor of node i
        for (int j = 1; j < LOG; j++) {
            for (int i = 1; i <= n; i++) {
                int mid = up[i][j - 1];
                if (mid != -1) {
                    up[i][j] = up[mid][j - 1];
                }
            }
        }
        // Answer queries in O(log k)
        while (q--) {
            int u, k;
            cin >> u >> k;
            int current = u;
            for (int i = 0; i < LOG; i++) {
                if ((k >> i) & 1) {
                    current = up[current][i];
                    if (current == -1) break;
                }
            }
            cout << current << "\n";
        }
    }

    int main() {
        // Faster I/O is necessary for 200,000 queries
        ios_base::sync_with_stdio(false);
        cin.tie(NULL);
        int t;
        if (!(cin >> t)) return 0;
        while (t--) {
            solve();
        }
        return 0;
    }
}

```

Trie

```

#include<bits/stdc++.h>
using namespace std;
class Trie {
    static const int N = 26;
    static const char baseChar = 'a';
    struct TrieNode {

        int next[N];
        bool isEnd;
        int freq;
        TrieNode() {
            memset(next, -1, sizeof(next));
            isEnd = false; freq = 0; };
    public:
        vector<TrieNode> tree;
        Trie() { tree.emplace_back(); }
        void insert(const string &s) {
            int p = 0; tree[p].freq++;
            for(char c : s) {
                int idx = c - baseChar;
                if(tree[p].next[idx] == -1) {
                    tree.emplace_back();
                    tree[p].next[idx] = tree.size() - 1;
                    p = tree[p].next[idx];
                    tree[p].freq++;
                }
                tree[p].isEnd = true;
            }

            bool checkPrefix(const string &s) {
                int p = 0;
                for(char c : s) {
                    if((p = tree[p].next[c - baseChar]) == -1) return
                    false;}return true;}
            bool checkString(const string &s) {
                int p = 0;
                for(char c : s) {
                    if((p = tree[p].next[c - baseChar]) == -1) return
                    false; }
                return tree[p].isEnd;
            }
        };
};

```

String input functions

```

string s;
getline(cin, s);
// CRITICAL: If you used 'cin >> n' before getline,
// there's a leftover '\n' in the buffer. Clear it:
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(),
'\n');
getline(cin, s);

string line = "Competitive programming is fun";
stringstream ss(line);
string word;
while(ss >> word) {
    // processes "Competitive", then "programming",
    etc.}
// Reads from cin until character 'c' is hit

void readUntil(char delim) {
    string s;
    getline(cin, s, delim);
}

```